

Tema 03 — Programación basada en lenguaje de marcas y código embebido

IES Ntra. Sra. de los Remedios (Ubrique) · 2º DAW · Desarrollo Web en Entorno Servidor · Curso 2026/2027

 **Idea central del tema:** hasta ahora tu PHP «imprimía cosas». A partir de aquí tu código **toma decisiones, repite tareas, guarda colecciones de datos, se organiza en funciones y conversa con el usuario**. Es el salto de «página con trucos» a **aplicación web**.

Índice

- [Introducción y objetivos](#)
- [Marco normativo y currículo](#)
- [Mapa del tema](#)
- [Convenciones y leyenda de recuadros](#)

- 1. [Sentencias condicionales](#)
- 2. [Sentencias iterativas o bucles](#)
- 3. [Tipos de datos complejos: arrays](#)
- 4. [Principios de subprogramación](#)
- 5. [Funciones predefinidas del lenguaje PHP](#)
- 6. [Acceso a la información del cliente web](#)
- 7. [Novedades de PHP 8 que debes conocer](#)
- 8. [Proyecto integrador: «Mini-tienda»](#)
- 9. [Errores típicos y cómo depurarlos](#)
- 10. [Actividades y autoevaluación](#)

- [Resumen del tema](#)

Introducción

El tema anterior mostró la sintaxis básica de los lenguajes de programación del entorno servidor. Para dominar la programación web hacen falta además otros elementos:

- **Estructuras de control** (condicionales y bucles), que dan flexibilidad a las aplicaciones.
- **Subprogramación:** definir y utilizar funciones y procedimientos, propios del programador o predefinidos en el lenguaje.
- **Estructuras de datos complejas:** arrays (o matrices).
- **Comunicación con el cliente web:** recuperar la información que envía el usuario, por ejemplo desde un formulario.

Con estos elementos una aplicación web puede gestionar las peticiones del cliente.

Objetivos

- Utilizar mecanismos de decisión y bucles en bloques de sentencias embebidos en HTML.
- Almacenar y recuperar conjuntos de datos con arrays.
- Crear y utilizar funciones, propias y predefinidas, y organizarlas en librerías.
- Usar formularios web y recuperar la información enviada con los métodos GET y POST.
- Añadir comentarios al código y aplicar buenas prácticas de seguridad.

Marco normativo y currículo

Este tema corresponde al módulo profesional **0613 «Desarrollo web en entorno servidor»** del ciclo de **Técnico Superior en Desarrollo de Aplicaciones Web**, regulado por el **Real Decreto 686/2010, de 20 de mayo**, con la redacción dada por el **Real Decreto 405/2023**. Cada comunidad autónoma concreta el currículo con su propia orden (en Andalucía, la orden del ciclo DAW).

El tema trabaja el resultado de aprendizaje **RA3**, y refuerza criterios del **RA2**:

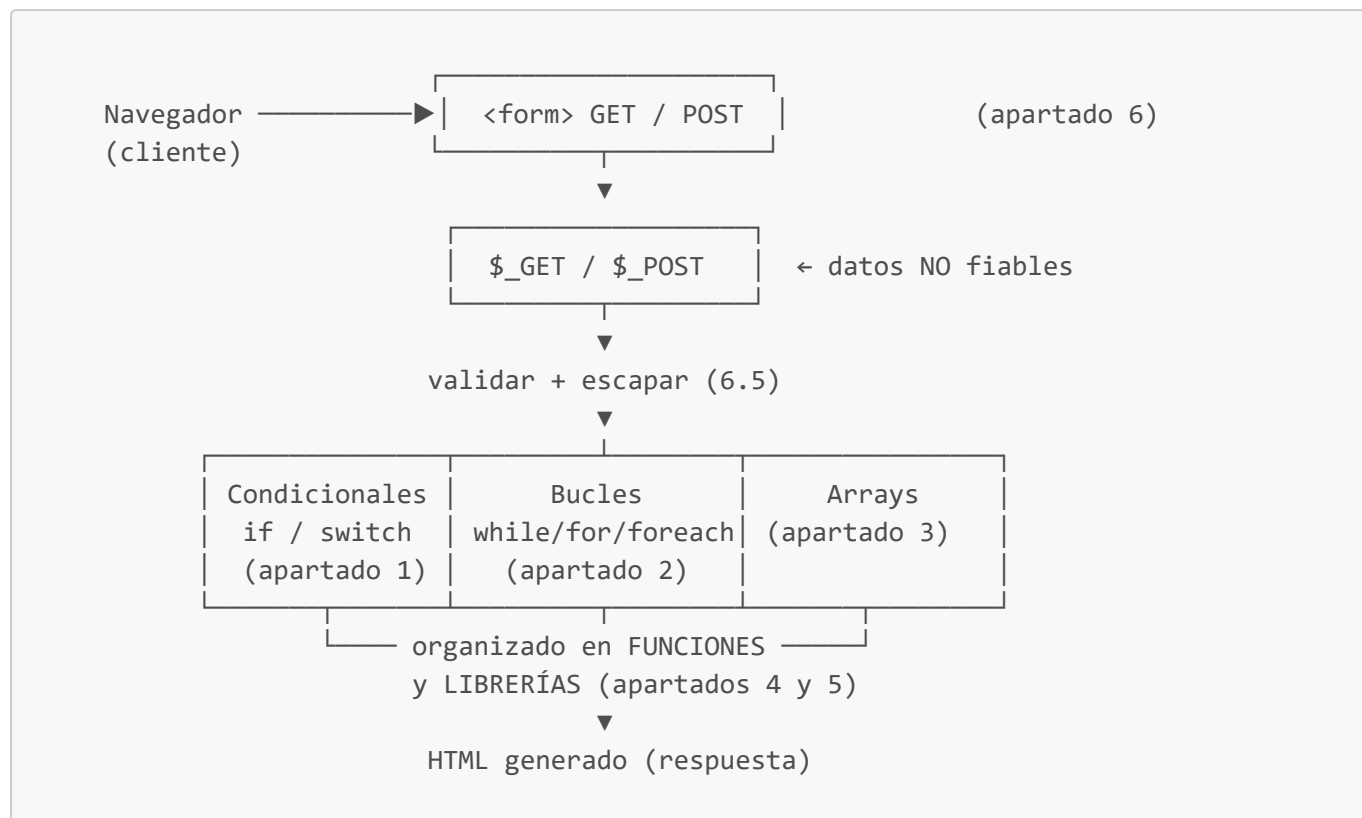
RA3. Escribe bloques de sentencias embebidos en lenguajes de marcas, seleccionando y utilizando las estructuras de programación.

Criterio de evaluación	Contenido del tema	Apartado
RA3.a) Se han utilizado mecanismos de decisión en la creación de bloques de sentencias.	<code>if, else, elseif, switch, match</code> , operador ternario	1
RA3.b) Se han utilizado bucles y se ha verificado su funcionamiento.	<code>while, do...while, for, foreach</code>	2
RA3.c) Se han utilizado matrices (arrays) para almacenar y recuperar conjuntos de datos.	Arrays escalares, asociativos y multidimensionales	3
RA3.d) Se han creado y utilizado funciones.	Funciones, argumentos, retorno, librerías, funciones predefinidas	4 y 5
RA3.e) Se han utilizado formularios web para interactuar con el usuario del navegador web.	Formularios HTML y envío de datos	6
RA3.f) Se han empleado métodos para recuperar la información introducida en el formulario.	<code>\$_GET, \$_POST, \$_REQUEST</code>	6
RA3.g) Se han añadido comentarios al código.	Comentarios y documentación (ver Tema 02)	Todo el tema
RA2.f) Se han utilizado directivas para modificar el comportamiento predeterminado.	<code>include, require</code> y sus variantes	4.6
RA2.g) Se han utilizado los distintos tipos de variables y operadores disponibles en el lenguaje.	Operadores de comparación y lógicos	1.1

Nota: los criterios están citados de forma resumida. Deben contrastarse con la versión consolidada vigente en el BOE y con la orden autonómica aplicable al centro.

Además, al trabajar con formularios se aplica la normativa de protección de datos (RGPD, LOPDGDD y LSSI-CE), que se resume en el apartado 6.5.

Mapa del tema



Convenciones del tema

Los ejemplos están escritos en **PHP** y, cuando procede, se comparan con **ASP clásico** y **JSP** para que se vean las diferencias entre lenguajes de servidor. Las notas con la etiqueta *Actualización* indican cambios de las versiones modernas de PHP (7.x y 8.x), resumidas en el apartado 7.

Icono	Significado
	Truco o idea clave
	Error frecuente o peligro
	Seguridad
	Novedad de PHP 8.x
	Pruébalo tú
	Para pensar
	Normativa o currículo

Los bloques de código con la etiqueta **php** son ejemplos ejecutables; los marcados como **text** son esquemas o salidas.

1. Sentencias condicionales

Las **sentencias condicionales** son estructuras de control que deciden el **flujo de ejecución** del programa, es decir, el orden en el que se ejecutan las instrucciones.

Sin condicionales, el código se ejecuta de forma secuencial, una instrucción detrás de otra. Con ellas, el programador define **caminos alternativos** según se cumplan o no ciertas condiciones.

Las más habituales son **if** y **switch**, que existen en ASP, PHP y JSP. Antes de verlas hay que saber cómo se construyen las condiciones.

1.1. Condiciones en PHP

Una condición es una expresión cuyo resultado es un valor lógico (**true** o **false**). Se construye con operadores de **comparación** y **lógicos**.

Operadores de comparación

Relacionan dos valores y devuelven un resultado lógico.

Ejemplo	Nombre	Resultado
<code>\$a == \$b</code>	Igual	TRUE si <code>\$a</code> es igual a <code>\$b</code> después de la manipulación de tipos.
<code>\$a === \$b</code>	Idéntico	TRUE si <code>\$a</code> es igual a <code>\$b</code> y son del mismo tipo .
<code>\$a != \$b</code>	Diferente	TRUE si <code>\$a</code> no es igual a <code>\$b</code> después de la manipulación de tipos.
<code>\$a <> \$b</code>	Diferente	Equivale a <code>!=</code> .
<code>\$a !== \$b</code>	No idéntico	TRUE si <code>\$a</code> no es igual a <code>\$b</code> , o si no son del mismo tipo.
<code>\$a < \$b</code>	Menor que	TRUE si <code>\$a</code> es estrictamente menor que <code>\$b</code> .
<code>\$a > \$b</code>	Mayor que	TRUE si <code>\$a</code> es estrictamente mayor que <code>\$b</code> .
<code>\$a <= \$b</code>	Menor o igual que	TRUE si <code>\$a</code> es menor o igual que <code>\$b</code> .
<code>\$a >= \$b</code>	Mayor o igual que	TRUE si <code>\$a</code> es mayor o igual que <code>\$b</code> .
<code>\$a <=> \$b</code>	Nave espacial (PHP 7)	Devuelve <code>-1</code> , <code>0</code> o <code>1</code> según <code>\$a</code> sea menor, igual o mayor que <code>\$b</code> .

```
<?php
// Variables de partida
$a = 10;
$b = '10';
$c = 5;
$d = 'Hola Pepe';
$e = 'Hola Luis';
$f = 'hola';

// Comprobamos las expresiones
$a == $b;      // true: son iguales
$a === $b;    // false: iguales, pero de distinto tipo
$a !== $b;    // true: $a es de distinto tipo que $b
$b > $c;      // true: $b es mayor que $c
$a != $c;     // true: $a es distinto de $c
$a <> $c;     // true: igual que la anterior
$d == $e;     // false: no son cadenas idénticas
```

```
$d[0] == $e[0]; // true: su primer carácter es idéntico
$d[0] == $f;    // false: distingue mayúsculas de minúsculas

$Resultado = ($a > $c) ? 'Es Mayor' : 'Es Menor';
echo $Resultado; // "Es Mayor", porque $a es mayor que $c
?>
```

Recomendación: usa `===` y `!==` siempre que puedas. La comparación laxa (`==`) puede dar resultados sorprendentes por la conversión automática de tipos (por ejemplo, `0 == "a"` fue `true` hasta PHP 7 y es `false` desde PHP 8).

Operadores lógicos

También llamados **booleanos**. Permiten crear **condiciones compuestas** y devuelven `true` o `false` según las tablas de verdad.

Ejemplo	Nombre	Resultado
<code>\$a and \$b</code>	And (y)	<code>TRUE</code> si tanto <code>\$a</code> como <code>\$b</code> son <code>TRUE</code> .
<code>\$a && \$b</code>	And (y)	Igual que <code>and</code> , pero con mayor precedencia .
<code>\$a or \$b</code>	Or (o inclusivo)	<code>TRUE</code> si cualquiera de <code>\$a</code> o <code>\$b</code> es <code>TRUE</code> .
<code>\$a \$b</code>	Or (o inclusivo)	Igual que <code>or</code> , pero con mayor precedencia .
<code>\$a xor \$b</code>	Xor (o exclusivo)	<code>TRUE</code> si <code>\$a</code> o <code>\$b</code> es <code>TRUE</code> , pero no ambos.
<code>! \$a</code>	Not (no)	<code>TRUE</code> si <code>\$a</code> no es <code>TRUE</code> .

Consejo: usa `&&`, `||` y `!` y añade paréntesis en las condiciones complejas para no depender de la precedencia.

1.2. Sentencia `if`

Define **dos flujos de ejecución** según se cumpla o no una condición:

- Si se cumple, se ejecuta una instrucción o un grupo de instrucciones.
- Si no se cumple, se ejecutan otras distintas (parte `else`, opcional).

`if simple` (solo se actúa si la condición es cierta):

```
if (condicion) {
    instrucciones;
}
```

`if ... else` (se indica qué hacer también cuando la condición es falsa):

```
if (condicion) {
    instrucciones_si_cierta;
```

```

} else {
    instrucciones_si_falsa;
}

```

if anidados (evalúan varias condiciones antes de ejecutar las instrucciones):

```

if (condicion1) {
    instrucciones1;
} elseif (condicion2) {
    instrucciones2;
} else {
    instrucciones3;
}

```

Comparativa de sintaxis entre lenguajes

	PHP	JSP (Java)	ASP
if	if (cond) { ... }	if (cond) { ... }	If cond Then ... End If
else	else { ... }	else { ... }	Else
Anidado	elseif o else if	Solo else if (separado)	ElseIf o Else If

if abreviado (operador ternario)

Reduce el código al mínimo, aunque se pierde algo de claridad en la estructura lógica. La estructura:

```

(condicion) ? instruccion1 : instruccion2;

```

es equivalente a un **if ... else**. Ejemplo con la sintaxis completa:

```

if ($a > $b) {
    $resultado = "A es Mayor que B";
} else {
    $resultado = "B es Mayor que A";
}

```

Con el método abreviado:

```

$resultado = ($a > $b) ? "A es Mayor que B" : "B es Mayor que A";

```

Actualización: el operador de fusión de *null* ?? (PHP 7) devuelve el primer operando si existe y no es *null*: `$nombre = $_GET['nombre'] ?? 'Anónimo'`; . El operador ?: abreviado devuelve el primer operando si es «verdadero».

Sintaxis alternativa del `if`

Se usa para crear **plantillas HTML**, es decir, para combinar bloques de HTML con código PHP sin llaves. Cada estructura se abre con `:` y se cierra con `endif;`.

```
<?php if (condicion): ?>
    Comandos HTML 1
<?php else: ?>
    Comandos HTML 2
<?php endif; ?>
```

También puede usarse solo con código PHP:

```
<?php
if (condicion-1):
    bloque-instrucciones-1;
elseif (condicion-2):
    bloque-instrucciones-2;
else:
    bloque-instrucciones-3;
endif;
?>
```

Ejemplos:

```
<?php if ($a == 5): ?>
    A es igual a 5
<?php endif; ?>

<?php if ($a == 5): ?>
    A es igual a 5
<?php else: ?>
    A no es igual a 5
<?php endif; ?>

<?php if ($a == 5): ?>
    A es igual a 5
<?php elseif ($a > 5): ?>
    A es mayor que 5
<?php else: ?>
    A es menor que 5
<?php endif; ?>
```

```
<?php
if ($a == 5):
    echo "a igual 5";
    echo "...";
elseif ($a == 6):
    echo "a igual 6";
    echo "!!!";
else:
    echo "a no es 5 ni 6";
endif;
?>
```

El código resulta más legible cuando se mezcla mucho HTML con PHP.

Importante: con la sintaxis alternativa debe escribirse **elseif** en una sola palabra; **else if** separado no está permitido.

1.3. Sentencia **switch** (o **select case**)

Se usa cuando, según el valor de una variable o expresión, hay que ejecutar un conjunto de instrucciones distinto para cada valor posible.

Funcionamiento:

1. Se calcula el valor de la expresión.
2. Se compara con cada **case**, en orden.
3. Cuando coincide, se ejecutan las instrucciones de ese caso.
4. Si ninguno coincide, se ejecuta el bloque **default**, si existe (es **opcional**).

Un **switch** suele sustituir con ventaja a una cadena de **if** anidados: **el código es más legible** y la expresión se evalúa **una sola vez**.

Comparativa de sintaxis entre lenguajes

Elemento	PHP	JSP (Java)	ASP
Inicio	switch (\$x) {	switch (x) {	Select Case x
Caso	case 1:	case 1:	Case 1 (sin :)
Fin de caso	break;	break;	No necesita break
Caso por defecto	default:	default:	Case Else
Fin de estructura	}	}	End Select

En PHP y JSP, **break** interrumpe la evaluación de los casos siguientes una vez encontrado el que coincide.

```
<?php
$forma_pago = 2;
switch ($forma_pago) {
    case 0:
```

```
    $pago = 'Contado';  
    break;  
case 1:  
    $pago = 'Transferencia bancaria';  
    break;  
case 2:  
    $pago = 'Contra reembolso';  
    break;  
default:  
    $pago = 'No definida';  
}  
?>
```

La lista de sentencias de un caso puede estar **vacía**: el control pasa al caso siguiente. Es útil para agrupar varios valores:

```
<?php  
switch ($i) {  
    case 0:  
    case 1:  
    case 2:  
        echo "i es menor que 3 pero no negativo";  
        break;  
    case 3:  
        echo "i es 3";  
}  
?>
```

Precaución: `switch` compara con `==` (comparación laxa), no con `===`. Si olvidas un `break`, la ejecución «cae» en el caso siguiente.

`switch` avanzado con condiciones

Un `switch` puede comportarse como una cadena `if ... elseif ... else` con condiciones simples o múltiples. Es útil cuando hay muchos casos con código simple o cuando los valores pertenecen a **rangos**.

La técnica es hacer `switch (true)`, de modo que cada `case` sea una condición que se evalúa a `true` o `false`.

Ejemplo con rangos de edad para la vida laboral:

```
<?php  
switch (true) {  
    case ($edad < 16):  
        echo 'Edad No Laboral';  
        break;  
    case ($edad >= 16 && $edad < 65):  
        echo 'Edad Laboral';  
        break;  
}
```

```
case ($edad >= 65):  
    echo 'Edad de jubilación';  
    break;  
}  
?>
```

match (PHP 8)

Actualización: PHP 8 incorpora la expresión `match`, parecida a `switch` pero **más segura**: compara con `===`, **devuelve un valor** y no necesita `break`. Si ningún caso coincide y no hay `default`, lanza `UnhandledMatchError`.

```
$pago = match ($forma_pago) {  
    0      => 'Contado',  
    1      => 'Transferencia bancaria',  
    2      => 'Contra reembolso',  
    default => 'No definida',  
};
```

Sintaxis alternativa del switch

Como en el `if`, sirve para integrar PHP en HTML y construir **plantillas**:

```
<?php switch ($var): ?>  
<?php case 1: ?>  
    Bloque HTML 1  
<?php break; ?>  
<?php case 2: ?>  
    Bloque HTML 2  
<?php break; ?>  
<?php default: ?>  
    Bloque HTML por defecto  
<?php endswitch; ?>
```

Nota: entre `switch` y el primer `case` no puede haber ninguna salida (ni siquiera espacios en blanco), o PHP dará un error. Por eso en la práctica se escribe `<?php switch ($var): ?>` seguido directamente de `<?php case 1: ?>`.

2. Sentencias iterativas o bucles

Los **bucles** ejecutan de forma repetida una instrucción o un grupo de instrucciones, un número **determinado** o **indeterminado** de veces según el tipo de bucle.

Este apartado define la sintaxis, el comportamiento y la utilidad de cada bucle en PHP, JSP y ASP:

Bucle	Cuándo se usa
<code>while</code>	Número de repeticiones indeterminado; se evalúa la condición antes de cada vuelta.
<code>do ... while</code>	Número indeterminado; se ejecuta al menos una vez , porque la condición se evalúa al final.
<code>for</code>	Número de repeticiones conocido, con un contador.
<code>foreach</code>	Recorrer los elementos de un array.

Como en los condicionales, la condición puede ser **simple** o **compuesta** por varias expresiones lógicas y aritméticas, y su resultado es un valor booleano.

2.1. Bucle `while`

Ejecuta el cuerpo un número indeterminado de veces **mientras la condición sea verdadera**.

1. En cada iteración se evalúa la condición.
2. Si es verdadera, se ejecuta el cuerpo del bucle.
3. El bucle termina cuando la condición es falsa.

```
while (condicion) {
    instrucciones;
}
```

	PHP	JSP	ASP
Sintaxis	<code>while (cond) { ... }</code>	<code>while (cond) { ... }</code>	<code>Do While cond ... Loop</code>

ASP admite además la forma antigua `While cond ... Wend`, un vestigio de los primeros tiempos de BASIC. Los intérpretes la siguen soportando, pero es posible que en el futuro dejen de hacerlo, por lo que se recomienda la forma nueva.

```
<?php
$Contador = 0;
echo '<p>';
while ($Contador <= 6) {
    echo "Contador con valor: $Contador<br>";
    $Contador++;
}
echo '</p>';
?>
```

Cuidado con los bucles infinitos: dentro del cuerpo debe modificarse alguna variable que afecte a la condición; si no, el bucle no termina y el servidor devolverá un error de *timeout* (ver errores de nombres de variable en el Tema 02).

Sintaxis alternativa (solo PHP):

```
while (expr):  
    sentencias;  
endwhile;
```

Ejemplo que muestra los números del 1 al 10:

```
<?php  
$i = 1;  
while ($i <= 10):  
    echo $i;  
    $i++;  
endwhile;  
?>
```

Mezclando código PHP con HTML:

```
<?php while (expr): ?>  
    Sentencias HTML  
<?php endwhile; ?>
```

2.2. Bucle `do ... while`

Es muy parecido al `while`, con una diferencia: **siempre se ejecuta al menos una vez**, se cumpla o no la condición, porque ésta se evalúa **al final** de cada iteración. Después se repite un número indeterminado de veces hasta que la condición es falsa.

```
do {  
    instrucciones;  
} while (condicion);
```

La sintaxis es idéntica en PHP y JSP. En ASP solo cambia el nombre del bucle (`Do ... Loop While cond`).

```
<?php  
$i = 0;  
do {  
    echo $i;  
    $i++;  
} while ($i < 5);    // muestra 01234  
?>
```

Nota: en el ejemplo del documento original, la condición era `$i > 0` con `$i = 0`, por lo que el cuerpo se ejecutaba una sola vez. Es un buen ejemplo de que `do ... while` entra en el bucle aunque la

condición sea falsa desde el principio.

2.3. Bucle **for**

A diferencia de los anteriores, se usa cuando el **número de repeticiones es conocido**. Se controla mediante un **contador**.

Funcionamiento en PHP y JSP:

1. Se **inicializa** el contador.
2. Se **evalúa la condición**. Si es falsa, el bucle termina.
3. Si es verdadera, se ejecuta el cuerpo.
4. Se **actualiza** el contador (incremento) y se vuelve al paso 2.

```
for (inicializacion; condicion; incremento) {  
    instrucciones;  
}
```

Las tres partes son **opcionales**: el contador puede crearse antes del bucle, y la condición y el incremento pueden definirse dentro de él.

Funcionamiento en ASP (For ... Next): no se define una condición. Se crea un contador con valor inicial y un **límite**. El bucle se ejecuta por última vez cuando el contador alcanza el límite. De forma opcional se indica el incremento (**Step**); por defecto es 1.

```
For i = 1 To 10 Step 1  
    Response.Write(i)  
Next
```

Ejemplos en PHP:

```
<?php  
/* ejemplo 1 */  
for ($i = 1; $i <= 10; $i++) {  
    echo $i;  
}  
  
/* ejemplo 2: sin expresión de condición, se sale con break */  
for ($i = 1; ; $i++) {  
    if ($i > 10) {  
        break;  
    }  
    echo $i;  
}  
  
/* ejemplo 3: sin ninguna de las tres expresiones */  
$i = 1;
```

```

for ( ; ; ) {
    if ($i > 10) {
        break;
    }
    echo $i;
    $i++;
}
?>

```

Sintaxis alternativa (solo PHP):

```

for (ini-contador; condicion; incremento):
    bloque-instrucciones;
endfor;

```

```

<?php
for ($i = 1; $i <= 10; $i++):
    echo $i;
endfor;
?>

```

Mezclando PHP con HTML:

```

<?php for ($i = 1; $i <= 10; $i++): ?>
    <p>Párrafo <?= $i ?></p>
<?php endfor; ?>

```

Sentencias **break** y **continue**

Sentencia	Efecto
break;	Termina el bucle por completo.
continue;	Salta a la siguiente iteración sin ejecutar el resto del cuerpo.

2.4. Bucle **foreach**

Se explica en el apartado 3.3, porque está diseñado para recorrer arrays.

3. Tipos de datos complejos: arrays

Un **array** (o **matriz**) permite almacenar un conjunto de datos. Una matriz, en sentido estricto, es un array multidimensional.

3.1. Definición y acceso

Los arrays son una construcción tradicional de los lenguajes de programación. Un array es un **conjunto ordenado de elementos**, cada uno identificado por un **índice** (su posición en la colección). En cada posición, el array contiene un valor.

- Pueden usarse tantos índices como se quiera, aunque lo habitual es el array **unidimensional**.
- La **longitud** del array se modifica de forma dinámica cuando se añade un elemento.

Definición e inicialización por lenguajes

	PHP	JSP (Java)	ASP
Array	<code>\$a = array(1, 2, 3);</code> o <code>\$a = [1, 2, 3];</code>	<code>int[] a = new int[3];</code> o <code>int[] a = {1, 2, 3};</code>	<code>Dim a(2)</code> y <code>a(0) = 1</code>
Matriz	<code>\$m = [[1, 2], [3, 4]];</code>	<code>int[][] m = {{1, 2}, {3, 4}};</code>	<code>Dim m(1, 1)</code> y <code>m(0, 0) = 1</code>
Acceso	<code>\$a[0]</code>	<code>a[0]</code>	<code>a(0)</code>
Imprimir	<code>echo</code>	<code>out.println</code>	<code>Response.Write</code>

Particularidades de cada lenguaje:

- **PHP.** El array se define por pares **clave => valor**. La clave hace referencia al índice y es **opcional**. La clave solo puede ser de tipo entero o cadena; el valor puede ser de cualquier tipo.
- **ASP.** Se usa **Dim**, el nombre de la variable y el tamaño entre paréntesis. El array empieza en 0 y toma como límite el número indicado, por lo que **Dim a(3)** almacena **4** elementos.
- **JSP.** Hay varias formas. Una crea la variable de tipo array y con **new** reserva memoria, guarda la referencia y después se inicializa accediendo por **corchetes** (no paréntesis como ASP). La otra declara e inicializa en una sola línea.

Aunque la sintaxis cambia, el comportamiento es el mismo.

Recorrer un array

Para acceder a todas las posiciones se usa un bucle **for** o **foreach**. En ASP y PHP, **foreach** (en ASP, **For Each**) se creó para esta tarea, aunque también puede usarse **for**.

Operaciones básicas con arrays en PHP

Operación	Código
Borrar un array entero	<code>unset(\$miarray);</code>
Borrar un elemento	<code>unset(\$miarray[0]);</code>
Añadir un elemento al final	<code>\$miarray[] = 8;</code>

3.2. Tipos de arrays

Arrays escalares (indexados)

Se accede a los elementos por un **índice numérico** que representa su posición, empezando en **0**. Se definen de dos formas:

```
$miArray = array(valor1, valor2, valor3, ...); // constructor array()
$miArray = [valor1, valor2, valor3, ...];      // notación de corchetes
```

```
// Array escalar de valores numéricos (constructor array())
$variable = array(10, 20, 30, 40, 50);
echo $variable[0]; // 10
echo $variable[3]; // 40

// Array escalar de cadenas (notación de corchetes)
$variable = ['Enero', 'Febrero', 'Marzo', 'Abril'];
echo $variable[0]; // 'Enero'
echo $variable[2]; // 'Marzo'

// Array escalar de valores combinados
$variable = ['Enero', 10, 'Marzo', 20, 'Abril'];
echo $variable[0]; // 'Enero'
echo $variable[3]; // 20
```

Arrays asociativos

Se accede a los elementos con **claves personalizadas** (números o cadenas). Cada elemento se escribe con la sintaxis **clave => valor**.

- Si la clave es una cadena, va **entre comillas** en el índice: `$clientes['nombre']`.
- No se puede acceder por índice numérico, salvo que la clave sea numérica.

```
$miArray = array(clv1 => valor1, clv2 => valor2, clv3 => valor3);
$miArray = [clv1 => valor1, clv2 => valor2, clv3 => valor3];
```

```
// Claves string y valores numéricos
$Notas = ['Luis' => 6, 'Carmen' => 6, 'Pedro' => 3, 'Rosa' => 8];
echo $Notas['Pedro']; // 3
$indice = 'Luis';
echo $Notas[$indice]; // 6 (acceso mediante una variable)

// Claves string y valores string
$Capitales = ['España' => 'Madrid', 'Francia' => 'Paris', 'Italia' => 'Roma',
'Alemania' => 'Berlin'];
echo $Capitales['España']; // 'Madrid'
echo $Capitales[0]; // Error: la clave 0 no existe (aviso de índice no
```

```
definido)

// Claves y valores mezclados
$Valores = ['Peso' => 65, 10 => 'Diez', 'Altura' => 1.75, 20 => 'Nombre'];
echo $Valores['Altura'];    // 1.75
echo $Valores[20];         // 'Nombre'
```

Arrays multidimensionales

Es un array cuyos **valores son otros arrays**. Para acceder a un elemento se indica el índice de cada dimensión, con **tantos pares de corchetes como dimensiones** tenga.

Tipo	Se accede como	Ejemplo de uso
Índices escalares	[índice][índice]	Matriz numérica
Índice escalar y subíndice asociativo	[índice][clave]	Lista de registros (alumnos)
Índice asociativo y subíndice escalar	[clave][índice]	Ciudades por país
Índices y subíndices asociativos	[clave][clave]	Datos por país

```
<?php
// Índices escalares
$matrizNumerica = [
    [2, 4, 6, 8],
    [5, 10, 15, 20],
    [10, 20, 30, 40]
];
echo $matrizNumerica[0][1];    // 4

// Índice asociativo y subíndice escalar
$ciudades = [
    'España' => ['Valencia', 'Madrid', 'Barcelona'],
    'Francia' => ['Paris', 'Marsella', 'Lion'],
    'Italia' => ['Roma', 'Nápoles', 'Venecia']
];
echo $ciudades['España'][1];   // Madrid

// Índice escalar y subíndices asociativos
$alumnos = [
    ['Nombre' => 'Luis', 'Edad' => 45, 'Sexo' => 'Hombre'],
    ['Nombre' => 'Carmen', 'Edad' => 40, 'Sexo' => 'Mujer'],
    ['Nombre' => 'Pedro', 'Edad' => 25, 'Sexo' => 'Hombre']
];
echo $alumnos[2]['Edad'];      // 25

// Índices y subíndices asociativos
$paises = [
    'España' => ['capital' => 'Madrid', 'idioma' => 'Español', 'poblacion' =>
46400000],
    'Francia' => ['capital' => 'Paris', 'idioma' => 'Frances', 'poblacion' =>
```

```
66415161],
    'Italia' => ['capital' => 'Roma', 'idioma' => 'Italiano', 'poblacion' =>
59801000]
];
echo $países['España']['idioma']; // Español
?>
```

3.3. El constructor `foreach`

`foreach` ofrece un modo sencillo de **iterar sobre arrays**. Solo funciona con arrays: da un error si se usa con otro tipo de dato o con una variable no inicializada.

Tiene dos sintaxis:

```
foreach ($array as $valor) {
    sentencias;
}

foreach ($array as $clave => $valor) {
    sentencias;
}
```

- **Primera forma.** En cada iteración, el valor del elemento actual se asigna a `$valor` y el puntero interno avanza una posición.
- **Segunda forma.** Además asigna la **clave** del elemento actual a `$clave`.
- Ambas admiten la **sintaxis alternativa** con `:`, cerrando con `endforeach;`.

```
<?php
$array = array(1, 2, 3, 4);
foreach ($array as $valor) {
    echo $valor;
    echo "<br>";
}
?>
```

```
<?php
$array = array(1, 2, 3, 4);
foreach ($array as $key => $valor) {
    echo "Índice: " . $key . " Valor: " . $valor;
    echo "<br>";
}
?>
```

Ejemplo con un array bidimensional de índices y subíndices asociativos:

```
<?php
$países = [
    'España' => ['capital' => 'Madrid', 'idioma' => 'Español', 'poblacion' =>
46400000],
    'Francia' => ['capital' => 'Paris', 'idioma' => 'Frances', 'poblacion' =>
66415161],
    'Italia' => ['capital' => 'Roma', 'idioma' => 'Italiano', 'poblacion' =>
59801000]
];

echo "<ul>";
foreach ($países as $nombre => $país) {
    echo "<li>";
    echo $nombre . ":";
    echo "<ul>";
    foreach ($país as $campo => $valor) {
        echo "<li>" . $campo . ":" . $valor . "</li>";
    }
    echo "</ul>";
    echo "</li>";
}
echo "</ul>";
?>
```

El resultado es:

- España:
 - capital: Madrid
 - idioma: Español
 - poblacion: 46400000
- Francia:
 - capital: Paris
 - idioma: Frances
 - poblacion: 66415161
- Italia:
 - capital: Roma
 - idioma: Italiano
 - poblacion: 59801000

Actualización: con `foreach ($array as &$valor)` el elemento se recibe **por referencia** y puede modificarse dentro del bucle. Después de este bucle conviene hacer `unset($valor);` para evitar efectos laterales.

3.4. Funciones más frecuentes para arrays

En la versión actual de PHP hay más de 70 funciones para trabajar con arrays (ver el [manual de PHP](#)). Las más utilizadas son:

Función	Descripción
<code>implode()</code> (alias <code>join()</code>)	Convierte un array en una cadena de texto.
<code>explode()</code>	Convierte una cadena en un array.
<code>str_split()</code>	Convierte una cadena en un array de caracteres.
<code>preg_split()</code>	Convierte una cadena en un array usando expresiones regulares.
<code>count()</code> (alias <code>sizeof()</code>)	Cuenta el número de elementos.
<code>array_push()</code>	Añade elementos al final del array.
<code>sort()</code> , <code>asort()</code> , <code>ksort()</code>	Ordenan el array con distinto criterio (por valor, manteniendo claves, o por clave).
<code>unset()</code>	Elimina elementos.
<code>var_dump()</code> , <code>var_export()</code> , <code>print_r()</code>	Muestran el contenido del array.
<code>shuffle()</code>	Desordena el array.
<code>array_merge()</code>	Une varios arrays en uno.
<code>array_search()</code>	Busca un valor en un array.
<code>array_rand()</code>	Devuelve una clave aleatoria.
<code>array_chunk()</code>	Divide un array en varios arrays.
<code>array_unique()</code>	Elimina los valores duplicados.
<code>array_keys()</code>	Devuelve un array con las claves de otro.

Nota: `foreach` no es una función, sino una construcción del lenguaje, aunque aparezca en muchas listas de recursos de arrays.

4. Principios de subprogramación

Este apartado explica cómo definir y usar **funciones** y **procedimientos** propios para fomentar la **reutilización de código** y la **programación modular**.

4.1. Definición y uso

Un **subprograma** es un fragmento de código con una funcionalidad específica. Permite que el código sea modular y reutilizable. Nacen de la idea de que un problema difícil es más fácil de resolver si se divide en otros más pequeños y sencillos.

Tipo	Qué hace
Función	Devuelve un valor como resultado de su ejecución.
Procedimiento	Ejecuta un conjunto de instrucciones sin devolver ningún valor.

Sintaxis de las funciones por lenguajes

	PHP	JSP (Java)	ASP
Definición	<code>function sumar(\$a, \$b) { return \$a + \$b; }</code>	<code>int sumar(int a, int b) { return a + b; }</code>	<code>Function sumar(a, b) ... sumar = a + b ... End Function</code>
Devolver valor	Sentencia <code>return</code>	Sentencia <code>return</code>	Se asigna el valor al nombre de la función
Tipos	No obligatorios (opcionales desde PHP 7)	Se indica el tipo de cada argumento y del valor devuelto	No se indican

Sintaxis de los procedimientos por lenguajes

	PHP	JSP (Java)	ASP
Definición	<code>function saludar(\$n) { echo "Hola \$n"; }</code>	<code>void saludar(String n) { out.println("Hola " + n); }</code>	<code>Sub saludar(n) ... End Sub</code>
Particularidad	Igual que una función, sin <code>return</code>	Se declara <code>void</code> como tipo devuelto	Sintaxis específica <code>Sub ... End Sub</code>

Dónde se definen y cómo se usan

- Las funciones y procedimientos suelen definirse en la **cabecera del documento**, para que estén cargados antes de usarlos.
- Para usarlos solo hay que **llamarlos por su nombre** pasando los parámetros necesarios.
- En el caso de las funciones se puede **asignar el valor devuelto** a una variable para usarlo después.

```
<?php
function sumar($a, $b) {
    return $a + $b;
}

$resultado = sumar(3, 4);    // llamada a la función
echo $resultado;            // 7
?>
```

Nota: en PHP, las funciones definidas en el nivel superior del fichero están disponibles aunque se llamen antes de su definición. Las definidas dentro de un `if` o de otra función solo existen cuando se ejecuta ese código.

4.2. Nombre de la función

Los nombres de función siguen las mismas reglas que el resto de identificadores de PHP: comienzan con una **letra o guion bajo**, seguidos de cualquier número de letras, números o guiones bajos.

Los nombres de función **no distinguen mayúsculas de minúsculas** (*case-insensitive*), a diferencia de los de variable. Aun así, por convención se escriben siempre igual que en su definición.

4.3. Argumentos

Se puede pasar información a las funciones mediante una **lista de argumentos** separados por comas.

- Se evalúan de **izquierda a derecha**, por lo que deben enviarse en el **mismo orden** en que se definieron.
- Los datos enviados deben ser del **tipo que espera la función**. Si espera dos números, se envían dos números; si espera una cadena y un número, se envían una cadena y un número. Si no, el resultado puede ser inesperado.

```
function NombreFuncion($Argumento1, $Argumento2) {  
    // líneas de código  
}
```

Actualización: PHP permite declarar tipos en parámetros y retorno: `function sumar(int $a, int $b): int { ... }`. Con `declare(strict_types=1)`; PHP 7 y 8 no convierten tipos de forma automática. PHP 8 permite además **argumentos con nombre** (`sumar(b: 4, a: 3)`).

Argumentos por valor y por referencia

Por defecto, los argumentos se pasan **por valor**: la función recibe una **copia**. Para pasarlos **por referencia**, se antepone `&` al nombre del argumento; así los cambios dentro de la función afectan a la variable original.

```
function NombreFuncion($Argumento1, &$Argumento2) {  
    // líneas de código  
}
```

Por valor:

```
function sumar_uno($x) {  
    $x++;  
    return $x;  
}  
$a = 2;  
echo sumar_uno($a);    // 3  
echo $a;               // 2: el original no cambia
```

Por referencia:

```
function sumar_uno(&$x) {  
    $x++;  
    return $x;  
}
```

```

}
$a = 2;
echo sumar_uno($a);    // 3
echo $a;               // 3: el cambio se refleja en la variable original

```

	Por valor	Por referencia
Qué recibe la función	Una copia	La propia variable
¿Modifica el original?	No	Sí
Símbolo	Ninguno	& antes del parámetro

PHP admite además listas de argumentos de **longitud variable** (por ejemplo, `function suma(...$numeros)`).

Argumentos con valores predeterminados

Se asigna un valor al argumento en la definición. Se usa cuando **no se recibe** ese argumento en la llamada.

- Los argumentos con valor predeterminado deben ir **a la derecha** de los que no lo tienen (los últimos de la lista).

```

function NombreFuncion($Argumento1, $Argumento2 = Valor, $Argumento3 = Valor) {
    // líneas de código
}

```

```

<?php
function foo($month = 8, $year = 2019) {
    return "mes $month de $year";
}
echo foo();           // mes 8 de 2019
echo foo(10);         // mes 10 de 2019
echo foo(1, 1900);    // mes 1 de 1900
?>

```

Arrays como argumentos

PHP admite arrays para declarar argumentos predeterminados, lo que permite enviar **múltiples valores en un solo argumento**.

Con un **array asociativo** (la clave es el nombre del argumento y el valor es su contenido) se pueden enviar los argumentos que se quieran **en cualquier orden**. Dentro de la función se usa `array_key_exists()` para saber cuáles se han recibido.

```
function NombreFuncion($Argumento = []) {
    if (array_key_exists('NombreArgumento', $Argumento)) {
        $Arg1 = $Argumento['NombreArgumento'];
    }
    // líneas de código
}
```

```
function eti_html($eti = []) {
    // Variables de trabajo para crear la etiqueta HTML
    $contenido = '';
    $etiqueta = 'p';
    $clase = '';

    // Si se reciben los argumentos esperados, se asigna su valor
    if (array_key_exists('contenido', $eti)) $contenido = $eti['contenido'];
    if (array_key_exists('etiqueta', $eti)) $etiqueta = $eti['etiqueta'];
    if (array_key_exists('clase', $eti)) $clase = 'class="' .
$eti['clase'] . '"';

    // Imprime la etiqueta HTML con su contenido y la clase CSS
    echo "<$etiqueta $clase>$contenido</$etiqueta>";
}

// Imprime un título de nivel 2 con las clases CSS tit-nav y text-center
$array['contenido'] = 'Titulo nivel 2';
$array['etiqueta'] = 'h2';
$array['clase'] = 'tit-nav text-center';
eti_html($array);
```

Seguridad: si el contenido procede del usuario, escápalo con `htmlspecialchars()` antes de imprimirlo.

4.4. Devolver valores

Una función devuelve un valor con la sentencia `return`, seguida del valor. Se puede devolver **cualquier tipo de dato**, incluidos arrays y objetos.

- `return` **finaliza la ejecución** de la función y devuelve el control a la instrucción que la llamó.
- Si se omite `return`, la función devuelve `NULL`.

```
function NombreFuncion($argumentos) {
    // líneas de código
    return $valorRetorno;
}
```

Actualización: PHP permite devolver varios valores empaquetados en un array (`return [$a, $b];`) y recogerlos con `[$x, $y] = funcion();`.

4.5. Librerías de funciones

Las funciones útiles para varios scripts se agrupan en un único archivo PHP, formando una **librería**.

Ventajas:

- No hay que reescribir las funciones en cada script: basta con **incluir la librería** al principio.
- Si se añade o modifica una función de la librería, el cambio se refleja en **todos los scripts** que la incluyen.

Para incluir una librería hay cuatro sentencias: `include`, `include_once`, `require` y `require_once`.

4.6. `include` y `require`

Incluyen en el script el contenido del archivo indicado y lo evalúan.

```
include 'ruta/archivo.php';
require 'ruta/archivo.php';
```

Los archivos se buscan según la ruta dada. Si no hay ruta, se buscan en el directorio actual o en la lista de directorios de la directiva `include_path` de PHP.

Sentencia	Si el archivo no existe	¿Se incluye varias veces?
<code>include</code>	Genera un aviso (<i>warning</i>) y la ejecución continúa.	Sí
<code>include_once</code>	Aviso, y continúa.	No: solo una vez.
<code>require</code>	Genera un error fatal y se interrumpe el código.	Sí
<code>require_once</code>	Error fatal.	No: solo una vez.

Las variantes `_once` evitan incluir el mismo código dos veces; si ya se había incluido, devuelven `TRUE`.

```
<?php
require_once 'funciones.php';
// ... resto del código
?>
```

Buena práctica: usa `require_once` para librerías de funciones imprescindibles y construye las rutas con `__DIR__` (`require_once __DIR__ . '/funciones.php';`). Nunca incluyas ficheros cuya ruta venga de datos del usuario: es una vulnerabilidad grave (*File Inclusion*).

4.7. Relación con el currículo

Las directivas de inclusión permiten **separar la presentación de la lógica** y evitar la dispersión del código (ver Tema 02). Esto enlaza con el criterio **RA2.f** (uso de directivas que modifican el comportamiento predeterminado) y con el RA5, que se estudiará más adelante.

5. Funciones predefinidas del lenguaje PHP

PHP incluye miles de funciones predefinidas. Este apartado resume las más útiles por categorías. La lista completa está en el [manual de PHP](#).

Importante: varias funciones de la lista original han quedado **obsoletas o eliminadas** en PHP 8 (`money_format`, `convert_cyr_string`, `hebrevc`, `each`) o están **desaconsejadas** (`strftime`, `strtotime`, `date_sunrise`, `date_sunset`, `crypt` con sal débil). Consulta siempre el manual de la versión que uses.

5.1. Funciones para cadenas (*string*)

Categoría	Funciones
Longitud y contenido	<code>strlen</code> , <code>str_word_count</code> , <code>count_chars</code> , <code>substr_count</code>
Mayúsculas y minúsculas	<code>strtolower</code> , <code>strtoupper</code> , <code>ucfirst</code> , <code>lcfirst</code> , <code>ucwords</code>
Búsqueda	<code>strpos</code> , <code>stripos</code> , <code>strrpos</code> , <code>strripos</code> , <code>strpos</code> , <code>stristr</code> , <code>strchr</code> , <code>strrchr</code> , <code>strpbrk</code> , <code>strspn</code> , <code>strcspn</code>
Extracción y modificación	<code>substr</code> , <code>substr_replace</code> , <code>str_replace</code> , <code>str_ireplace</code> , <code>strtr</code> , <code>str_pad</code> , <code>str_repeat</code> , <code>strrev</code> , <code>str_shuffle</code> , <code>wordwrap</code> , <code>chunk_split</code>
Espacios en blanco	<code>trim</code> , <code>ltrim</code> , <code>rtrim</code> (alias <code>chop</code>)
Comparación	<code>strcmp</code> , <code>strcasecmp</code> , <code>strncmp</code> , <code>strncasecmp</code> , <code>strnatcmp</code> , <code>strnatcasecmp</code> , <code>strcoll</code> , <code>substr_compare</code> , <code>similar_text</code> , <code>levenshtein</code> , <code>soundex</code> , <code>metaphone</code>
Conversión entre cadenas y arrays	<code>explode</code> , <code>implode</code> (alias <code>join</code>), <code>str_split</code> , <code>str_getcsv</code> , <code>strtok</code> , <code>parse_str</code>
Formato de salida	<code>echo</code> , <code>print</code> , <code>printf</code> , <code>sprintf</code> , <code>vprintf</code> , <code>vsprintf</code> , <code>fprintf</code> , <code>vfprintf</code> , <code>number_format</code> , <code>nl2br</code> , <code>sscanf</code>
Códigos de caracteres	<code>chr</code> , <code>ord</code> , <code>bin2hex</code> , <code>hex2bin</code>
HTML y seguridad	<code>htmlspecialchars</code> , <code>htmlspecialchars_decode</code> , <code>htmlentities</code> , <code>html_entity_decode</code> , <code>get_html_translation_table</code> , <code>strip_tags</code> , <code>addslashes</code> , <code>stripslashes</code> , <code>addcslashes</code> , <code>stripclslashes</code> , <code>quotemeta</code>
Resúmenes (<i>hash</i>) y cifrado	<code>md5</code> , <code>md5_file</code> , <code>sha1</code> , <code>sha1_file</code> , <code>crc32</code> , <code>crypt</code> , <code>str_rot13</code>

Categoría	Funciones
Codificación	<code>convert_uuencode</code> , <code>convert_uudecode</code> , <code>quoted_printable_encode</code> , <code>quoted_printable_decode</code>
Configuración regional	<code>setlocale</code> , <code>localeconv</code> , <code>nl_langinfo</code>

Seguridad con contraseñas: `md5()` y `sha1()` **no son adecuadas** para almacenar contraseñas. Usa `password_hash()` y `password_verify()`.

Ejemplo de funciones de uso diario:

```
$texto = "  Desarrollo Web en Entorno Servidor  ";
echo strlen($texto);           // 38
echo trim($texto);             // "Desarrollo Web en Entorno Servidor"
echo strtoupper(trim($texto)); // "DESARROLLO WEB EN ENTORNO SERVIDOR"
echo str_replace("Web", "web", $texto); // cambia "Web" por "web"
echo substr(trim($texto), 0, 10); // "Desarrollo"
echo strpos($texto, "Web");     // posición de la primera aparición
```

Nota: en PHP 8 se añadieron `str_contains()`, `str_starts_with()` y `str_ends_with()`, más claras que `strpos()` para comprobar si una cadena contiene a otra. Para texto con tildes conviene usar las funciones `mb_*` (`mb_strlen`, `mb_substr`, `mb_strtoupper`), compatibles con UTF-8.

5.2. Funciones para arrays

Categoría	Funciones
Creación y relleno	<code>array</code> , <code>array_fill</code> , <code>array_fill_keys</code> , <code>array_pad</code> , <code>range</code> , <code>array_combine</code> , <code>compact</code> , <code>extract</code> , <code>list</code>
Añadir y quitar elementos	<code>array_push</code> , <code>array_pop</code> , <code>array_shift</code> , <code>array_unshift</code> , <code>array_splice</code> , <code>array_slice</code>
Claves y valores	<code>array_keys</code> , <code>array_values</code> , <code>array_key_exists</code> (alias <code>key_exists</code>), <code>array_key_first</code> , <code>array_key_last</code> , <code>array_flip</code> , <code>array_column</code> , <code>array_change_key_case</code> , <code>array_count_values</code>
Búsqueda	<code>in_array</code> , <code>array_search</code>
Unión y sustitución	<code>array_merge</code> , <code>array_merge_recursive</code> , <code>array_replace</code> , <code>array_replace_recursive</code>
Diferencia	<code>array_diff</code> , <code>array_diff_key</code> , <code>array_diff_assoc</code> , <code>array_diff_ukey</code> , <code>array_diff_uassoc</code> , <code>array_udiff</code> , <code>array_udiff_assoc</code> , <code>array_udiff_uassoc</code>
Intersección	<code>array_intersect</code> , <code>array_intersect_key</code> , <code>array_intersect_assoc</code> , <code>array_intersect_ukey</code> , <code>array_intersect_uassoc</code> , <code>array_uintersect</code> , <code>array_uintersect_assoc</code> , <code>array_uintersect_uassoc</code>

Categoría	Funciones
Ordenación	<code>sort</code> , <code>rsort</code> , <code>asort</code> , <code>arsort</code> , <code>ksort</code> , <code>krsort</code> , <code>usort</code> , <code>uasort</code> , <code>uksort</code> , <code>natsort</code> , <code>natcasesort</code> , <code>array_multisort</code> , <code>array_reverse</code> , <code>shuffle</code>
Funciones de orden superior	<code>array_map</code> , <code>array_filter</code> , <code>array_reduce</code> , <code>array_walk</code> , <code>array_walk_recursive</code>
Cálculo	<code>count</code> (alias <code>sizeof</code>), <code>array_sum</code> , <code>array_product</code> , <code>array_unique</code>
Aleatoriedad y partición	<code>array_rand</code> , <code>array_chunk</code>
Puntero interno	<code>current</code> (alias <code>pos</code>), <code>key</code> , <code>next</code> , <code>prev</code> , <code>reset</code> , <code>end</code>

Actualización: la función `each()` se eliminó en PHP 8. Para recorrer un array utiliza `foreach`.

```
$numeros = [5, 3, 8, 1];
sort($numeros);           // [1, 3, 5, 8]
echo count($numeros);     // 4
echo array_sum($numeros); // 17
echo in_array(8, $numeros) ? "sí" : "no"; // sí
$dobles = array_map(fn($n) => $n * 2, $numeros); // [2, 6, 10, 16]
```

5.3. Funciones de fecha y hora

Categoría	Funciones
Fecha y hora actual	<code>time</code> , <code>microtime</code> , <code>gettimeofday</code> , <code>date</code> , <code>gmdate</code> , <code>getdate</code> , <code>localtime</code> , <code>idate</code>
Marcas de tiempo (<i>timestamp</i>)	<code>mktime</code> , <code>gmmktime</code> , <code>strtotime</code> , <code>checkdate</code>
Zona horaria	<code>date_default_timezone_get</code> , <code>date_default_timezone_set</code> , <code>timezone_identifiers_list</code> , <code>timezone_open</code> , <code>timezone_name_get</code> , <code>timezone_offset_get</code> , <code>timezone_name_from_abbr</code> , <code>timezone_abbreviations_list</code> , <code>timezone_location_get</code> , <code>timezone_transitions_get</code> , <code>timezone_version_get</code>
Análisis de fechas	<code>date_parse</code> , <code>date_parse_from_format</code> , <code>date_get_last_errors</code>
Orientadas a objetos (alias de la clase <code>DateTime</code>)	<code>date_create</code> , <code>date_create_immutable</code> , <code>date_create_from_format</code> , <code>date_create_immutable_from_format</code> , <code>date_add</code> , <code>date_sub</code> , <code>date_diff</code> , <code>date_format</code> , <code>date_modify</code> , <code>date_date_set</code> , <code>date_time_set</code> , <code>date_isodate_set</code> , <code>date_timestamp_get</code> , <code>date_timestamp_set</code> , <code>date_timezone_get</code> , <code>date_timezone_set</code> , <code>date_offset_get</code> , <code>date_interval_create_from_date_string</code> , <code>date_interval_format</code>

Categoría	Funciones
Astronomía	<code>date_sun_info</code> , <code>date_sunrise</code> , <code>date_sunset</code>
Obsoletas (formato por configuración local)	<code>strftime</code> , <code>gmstrftime</code> , <code>strptime</code> (desaconsejadas desde PHP 8.1)

Las funciones `date_*` son alias de los métodos de las clases `DateTime`, `DateTimeImmutable`, `DateInterval` y `DateTimeZone`, que es la forma recomendada de trabajar con fechas en PHP moderno.

```
date_default_timezone_set('Europe/Madrid');
echo date('d/m/Y H:i:s');           // fecha y hora actuales
var_dump(checkdate(2, 30, 2026));   // false: el 30 de febrero no
existe                               existe

$hoy = new DateTime();
$limite = new DateTime('2026-12-31');
echo $hoy->diff($limite)->days;    // días hasta fin de año
```

Carácter de <code>date()</code>	Significado
<code>d / j</code>	Día del mes con / sin cero inicial
<code>m / n</code>	Mes con / sin cero inicial
<code>Y / y</code>	Año con 4 / 2 dígitos
<code>H / i / s</code>	Hora (24 h) / minutos / segundos

5.4. Funciones matemáticas

Categoría	Funciones
Valor y signo	<code>abs</code> , <code>max</code> , <code>min</code>
Redondeo y división	<code>round</code> , <code>ceil</code> , <code>floor</code> , <code>intdiv</code> , <code>fmod</code>
Potencias y raíces	<code>pow</code> , <code>sqrt</code> , <code>hypot</code> , <code>exp</code> , <code>expm1</code>
Logaritmos	<code>log</code> , <code>log10</code> , <code>log1p</code>
Trigonometría	<code>sin</code> , <code>cos</code> , <code>tan</code> , <code>asin</code> , <code>acos</code> , <code>atan</code> , <code>atan2</code> , <code>sinh</code> , <code>cosh</code> , <code>tanh</code> , <code>asinh</code> , <code>acosh</code> , <code>atanh</code> , <code>deg2rad</code> , <code>rad2deg</code> , <code>pi</code>
Cambio de base	<code>base_convert</code> , <code>bindec</code> , <code>decbin</code> , <code>hexdec</code> , <code>dechex</code> , <code>octdec</code> , <code>decoct</code>
Números aleatorios	<code>rand</code> , <code>mt_rand</code> , <code>mt_srand</code> , <code>srand</code> , <code>getrandmax</code> , <code>mt_getrandmax</code> , <code>lcg_value</code>

Categoría	Funciones
Comprobaciones	<code>is_finite</code> , <code>is_infinite</code> , <code>is_nan</code>

Seguridad: `rand()` y `mt_rand()` no son criptográficamente seguros. Para tokens o contraseñas temporales utiliza `random_int()` o `random_bytes()`.

```
echo round(3.14159, 2);    // 3.14
echo ceil(4.2);           // 5
echo floor(4.8);          // 4
echo max(3, 9, 5);        // 9
echo intdiv(17, 5);        // 3
echo mt_rand(1, 6);        // número aleatorio entre 1 y 6
```

6. Acceso a la información del cliente web

Este apartado explica los distintos tipos de paso de parámetros entre cliente y servidor, sus diferencias, cómo definir un formulario y cómo recuperar los datos enviados por **GET** y **POST**.

6.1. Métodos GET y POST

Ambos son **métodos del protocolo HTTP** para intercambiar información entre cliente y servidor.

Método GET

Se usa para **recuperar información**. Pide al servidor que devuelva el recurso identificado en la petición (URI): normalmente un documento HTML o una imagen, pero también puede ser el resultado de una consulta a una base de datos.

Los datos viajan **codificados en la propia URL**. La página y los datos se separan con `?`, y cada pareja `variable=valor` con `&`:

```
www.ejemplo.com/index.php?key1=value1&key2=value2&key3=value3
```

Características:

- La información se envía **en la URL**, con una longitud limitada (en la práctica, en torno a 2000 caracteres, según navegador y servidor).
- Los datos son **visibles**, por lo que **nunca debe enviarse información sensible**.
- **No** se pueden enviar datos binarios (archivos, imágenes...).
- En PHP se recogen en el array asociativo `$_GET`.
- Las peticiones GET pueden ser **cacheadas**, **indexadas** por los buscadores y **guardadas como favoritos**.

Método POST

Se usa para **enviar información** al servidor: completar un formulario de autenticación, introducir datos o especificar parámetros para algún software del servidor.

Los datos viajan **en el cuerpo** de la petición HTTP, por lo que no aparecen en la URL.

Características:

- **No tiene límite práctico** de cantidad de información (lo limita la configuración del servidor, por ejemplo `post_max_size`).
- La información **no es visible** en la URL.
- Puede enviar texto y también **datos binarios** (archivos, imágenes...).
- En PHP se recogen en el array asociativo `$_POST`.
- Las respuestas **no se cachean** ni se guardan como favoritos de forma habitual.

Comparativa GET frente a POST

Característica	GET	POST
Finalidad	Recuperar información	Enviar información
Dónde viajan los datos	En la URL	En el cuerpo de la petición HTTP
Visibilidad	Visibles	No visibles en la URL
Límite de tamaño	Sí (≈ 2000 caracteres)	Prácticamente ninguno
Datos binarios	No	Sí
Caché, favoritos y buscadores	Sí	No
Volver atrás con el botón del navegador	Sin problema	Puede pedir reenviar el formulario
Array de PHP	<code>\$_GET</code>	<code>\$_POST</code>
Uso típico	Búsquedas, filtros, paginación	Login, altas, modificaciones

Idea clave: si se usara GET para autenticarse o modificar datos, cualquiera podría manipular la URL o pasársela a otra persona. Por eso se usa **POST** para formularios que insertan o actualizan información.

Partes de una URL

Para entender cómo viajan las variables en GET hay que conocer las partes de una URL:

Parte	Descripción	Ejemplo
Protocolo	Protocolo de comunicación.	<code>https://</code>
Nombre de dominio	Nombre del servidor donde se aloja la información.	<code>www.ejemplo.com</code>
Directorios	Secuencia de directorios separados por <code>/</code> que indica la ruta del recurso.	<code>/cursos/dwes/</code>

Parte	Descripción	Ejemplo
Fichero	Nombre del recurso al que se quiere acceder.	<code>index.php</code>
Cadena de consulta	Tras <code>?</code> : variables con su valor (<code>variable=valor</code>), separadas por <code>&</code> .	<code>? nombre=Ana&curso=2DAW</code>

Ejemplo completo: <https://www.ejemplo.com/cursos/dwes/index.php?nombre=Ana&curso=2DAW>

¿Es más seguro POST que GET?

Se suele pensar que POST es más seguro que GET, pero **ninguno es seguro por sí mismo**:

- POST oculta los datos en la URL, pero **no los cifra**. Para proteger los datos en tránsito hay que usar **HTTPS**.
- Ambos métodos pueden recibir datos manipulados y sufrir ataques como la **inyección de HTML/JavaScript (XSS)** o la **inyección SQL**.

Conclusión: usa cada método para lo que fue creado y **valida siempre en el servidor** todo dato recibido.

- **GET** → recuperación de información.
- **POST** → envío de información.

6.2. Formularios HTML

Un formulario HTML envía datos al servidor. Los atributos principales de la etiqueta `<form>` son:

Atributo	Función
<code>action</code>	URL del script PHP que procesará los datos.
<code>method</code>	Método HTTP: <code>get</code> o <code>post</code> .
<code>enctype</code>	Tipo de codificación. Debe ser <code>multipart/form-data</code> para enviar archivos.

Cada campo se identifica con el atributo `name`, que será la **clave** del array que recibe PHP.

```
<form action="procesar.php" method="post">
  <label>Nombre: <input type="text" name="nombre"></label>
  <label>Apellidos: <input type="text" name="apellidos"></label>
  <input type="submit" value="Enviar">
</form>
```

6.3. Recuperación de la información con GET

Tras enviar el formulario, el servidor debe recuperar la información para procesarla. En PHP resulta sencillo: existe la variable especial `$_GET`, un array en el que:

- el **índice** es el valor del atributo `name` del elemento del formulario;
- el **valor** es lo que ha introducido el usuario.

```
<?php
// formulario con method="get" y campos "nombre" y "apellidos"
$nombre = $_GET['nombre'];
$apellidos = $_GET['apellidos'];
echo "Hola, $nombre $apellidos";
?>
```

6.4. Recuperación de la información con POST

Para el método POST se usa la variable `$_POST`. Es un array y almacena la información igual que `$_GET`: usa como índice el atributo `name` del elemento y guarda en su interior el valor.

La diferencia entre ambos métodos es **en qué parte del mensaje HTTP viajan los datos**: en la URL (GET) o en el cuerpo (POST).

```
<?php
// formulario con method="post" y campos "nombre" y "apellidos"
$nombre = $_POST['nombre'];
$apellidos = $_POST['apellidos'];
echo "Hola, $nombre $apellidos";
?>
```

PHP dispone también de `$_REQUEST`, que contiene los datos de `$_GET`, `$_POST` y `$_COOKIE`. Se puede usar para recuperar información de ambos métodos, pero **se desaconseja**: no se sabe de dónde procede cada dato y puede dar lugar a errores de seguridad.

Resumen de las variables superglobales de formularios

Variable	Contenido
<code>\$_GET</code>	Datos enviados por la URL (método GET).
<code>\$_POST</code>	Datos enviados en el cuerpo de la petición (método POST).
<code>\$_REQUEST</code>	<code>\$_GET</code> , <code>\$_POST</code> y <code>\$_COOKIE</code> combinados (desaconsejada).
<code>\$_FILES</code>	Archivos subidos mediante formulario (<code>enctype="multipart/form-data"</code>).
<code>\$_SERVER['REQUEST_METHOD']</code>	Método HTTP de la petición actual ('GET', 'POST' ...).

6.5. Validación y seguridad de los datos recibidos (ampliación)

Todo dato que llega del cliente es no confiable. El usuario (o un atacante) puede enviar cualquier valor, manipular la URL o saltarse el formulario. Estas buenas prácticas son imprescindibles:

1. **Comprobar que el dato existe** antes de usarlo (`isset()` u operador `??`), para evitar avisos de índice no definido.

2. **Validar** el tipo, el formato y el rango (`filter_input()`, `filter_var()`, `is_numeric()`...).
3. **Escapar** al mostrar los datos en HTML con `htmlspecialchars()`, para evitar ataques **XSS**.
4. **Usar consultas preparadas** al guardar datos en la base de datos (se estudiará en temas posteriores), para evitar la **inyección SQL**.
5. **Usar HTTPS** para proteger los datos en tránsito.
6. **No enviar datos sensibles por GET** (contraseñas, datos personales): quedan en el historial del navegador, en los registros del servidor y en los *referers*.
7. En formularios que modifiquen datos, usar **tokens CSRF** para evitar peticiones falsificadas.

Ejemplo de procesamiento seguro de un formulario:

```
<?php
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    $nombre = trim($_POST['nombre'] ?? '');
    $edad = filter_input(INPUT_POST, 'edad', FILTER_VALIDATE_INT);

    if ($nombre === '' || $edad === false || $edad === null) {
        echo "Datos incorrectos.";
    } else {
        echo "Hola, " . htmlspecialchars($nombre, ENT_QUOTES, 'UTF-8');
    }
}
?>
```

Protección de datos personales

Los formularios suelen recoger datos personales, por lo que se aplica la normativa de protección de datos:

Norma	Qué exige en relación con los formularios
Reglamento (UE) 2016/679 (RGPD)	Informar de la finalidad del tratamiento, recoger solo los datos necesarios (minimización), contar con una base jurídica (por ejemplo, el consentimiento) y aplicar medidas de seguridad adecuadas.
Ley Orgánica 3/2018 (LOPDGDD)	Adapta el RGPD al ordenamiento español y regula los derechos digitales.
Ley 34/2002 (LSSI-CE)	Regula los servicios de la sociedad de la información, incluida la información sobre cookies y las comunicaciones comerciales.

7. Novedades de PHP 8 que debes conocer

 PHP 8.0 (2020) y sus versiones posteriores (8.1, 8.2, 8.3...) incorporan mejoras que verás en código real. Resumen de las que afectan a este tema.

Novedad	Versión	Ejemplo	Apartado
Expresión <code>match</code>	8.0	<code>\$t = match(\$x) { 1 => 'a', default => 'b' };</code>	1.3

Novedad	Versión	Ejemplo	Apartado
Comparación número/cadena más lógica	8.0	<code>0 == "a" → false</code>	1.1
Argumentos con nombre	8.0	<code>sumar(b: 4, a: 3)</code>	4.3
Tipos unión	8.0	<code>function f(int float \$n)</code>	4.3
Operador <i>nullsafe</i>	8.0	<code>\$user?->perfil?->nombre</code>	—
<code>str_contains</code> , <code>str_starts_with</code> , <code>str_ends_with</code>	8.0	<code>str_contains("DWES", "WE")</code>	5.1
Promoción de propiedades en constructores	8.0	para POO (temas posteriores)	—
Funciones flecha	7.4	<code>fn(\$n) => \$n * 2</code>	5.2
Spread en arrays con claves de texto	8.1	<code>[...\$a, ...\$b]</code>	3
<code>array_is_list()</code>	8.1	<code>array_is_list([1, 2]) → true</code>	3
Enumeraciones (<code>enum</code>)	8.1	para POO (temas posteriores)	—
Propiedades <code>readonly</code>	8.1	para POO (temas posteriores)	—
<code>strftime</code> , <code>date_sunrise...</code> obsoletas	8.1	usar <code>DateTime</code> / <code>IntlDateFormatter</code>	5.3
<code>each()</code> , <code>money_format()</code> ... eliminadas	8.0	usar <code>foreach</code> / <code>NumberFormatter</code>	3, 5

Ejemplo: el mismo código en PHP 7 y PHP 8

```
<?php
// PHP 7: mucho código para decidir
switch ($codigo) {
    case 200: $texto = 'OK'; break;
    case 404: $texto = 'No encontrado'; break;
    default: $texto = 'Desconocido';
}

// PHP 8: expresión match, más corta y estricta
$texto = match ($codigo) {
    200      => 'OK',
    404      => 'No encontrado',
    default => 'Desconocido',
};

// PHP 7: buscar una subcadena
if (strpos($frase, 'PHP') !== false) { /* ... */ }
```

```
// PHP 8: más legible
if (str_contains($frase, 'PHP')) { /* ... */ }
```

Tipado estricto

```
<?php
declare(strict_types=1); // debe ser la primera instrucción del fichero

function doble(int $n): int
{
    return $n * 2;
}

echo doble(4); // 8
echo doble("4"); // TypeError: con strict_types no se convierte "4" a 4
```

💡 Recomendación para el curso: escribe **siempre** `declare(strict_types=1);` y tipa parámetros y retornos. Los errores aparecen antes y son más fáciles de entender.

8. Proyecto integrador: «Mini-tienda»

Reúne casi todo el tema: arrays, funciones, librerías, condicionales, bucles, GET y seguridad.

```
minitienda/
├─ index.php
├─ includes/
│   ├── datos.php
│   └─ funciones.php
```

`includes/datos.php` — los datos (en temas posteriores vendrán de una base de datos):

```
<?php
return [
    ['id' => 1, 'nombre' => 'Teclado mecánico', 'precio' => 59.90, 'cat' =>
'perifericos'],
    ['id' => 2, 'nombre' => 'Ratón inalámbrico', 'precio' => 19.95, 'cat' =>
'perifericos'],
    ['id' => 3, 'nombre' => 'Monitor 24"', 'precio' => 129.00, 'cat' =>
'pantallas'],
    ['id' => 4, 'nombre' => 'Webcam HD', 'precio' => 34.50, 'cat' =>
'perifericos'],
];
```

`includes/funciones.php` — la librería:

```

<?php
declare(strict_types=1);

/** Escapa un texto para mostrarlo en HTML (previene XSS). */
function e(?string $s): string
{
    return htmlspecialchars($s ?? '', ENT_QUOTES, 'UTF-8');
}

/** Da formato monetario español. */
function euros(float $n): string
{
    return number_format($n, 2, ',', '.') . ' €';
}

/** Filtra por categoría y precio máximo. */
function filtrar(array $productos, string $cat, ?float $max): array
{
    return array_filter($productos, function ($p) use ($cat, $max) {
        $okCat = ($cat === '' || $p['cat'] === $cat);
        $okMax = ($max === null || $p['precio'] <= $max);
        return $okCat && $okMax;
    });
}

```

index.php — la página:

```

<?php
declare(strict_types=1);
require_once __DIR__ . '/includes/funciones.php';
$productos = require __DIR__ . '/includes/datos.php';

// Lista blanca de categorías (seguridad)
$categorias = ['', 'perifericos', 'pantallas'];
$cat = $_GET['cat'] ?? '';
if (!in_array($cat, $categorias, true)) {
    $cat = '';
}

// Validación del precio máximo
$max = filter_var($_GET['max'] ?? '', FILTER_VALIDATE_FLOAT);
$max = ($max === false) ? null : (float) $max;

$lista = filtrar($productos, $cat, $max);
usort($lista, fn($a, $b) => $a['precio'] <=> $b['precio']);
?>

<!DOCTYPE html>
<html lang="es">
<head><meta charset="UTF-8"><title>Mini-tienda</title></head>
<body>

```

```

<h1>Mini-tienda</h1>

<form method="get">
  <select name="cat">
    <?php foreach ($categorias as $c): ?>
      <option value="<? e($c) ?>" <? $c === $cat ? 'selected' : '' ?>>
        <? $c === '' ? 'Todas' : e(ucfirst($c)) ?>
      </option>
    <?php endforeach; ?>
  </select>
  <input type="number" step="0.01" name="max" placeholder="Precio máx."
    value="<? $max !== null ? e((string) $max) : '' ?>"
  <button>Filtrar</button>
</form>

<?php if (count($lista) === 0): ?>
  <p>No hay productos con esos criterios.</p>
<?php else: ?>
  <ul>
    <?php foreach ($lista as $p): ?>
      <li><? e($p['nombre']) ?> - <strong><? euros($p['precio']) ?></strong></li>
    <?php endforeach; ?>
  </ul>
  <p>Total: <? count($lista) ?> productos.</p>
<?php endif; ?>
</body>
</html>

```

Concepto practicado	Dónde aparece
Librerías (<code>require_once</code>)	cabecera de <code>index.php</code>
Funciones tipadas y PHPDoc	<code>funciones.php</code>
Arrays multidimensionales	<code>datos.php</code>
<code>array_filter</code> , <code>usort</code> , closures	<code>filtrar()</code> y ordenación
GET y lista blanca	<code>\$_GET['cat']</code>
Validación (<code>filter_var</code>)	<code>\$_GET['max']</code>
Sintaxis alternativa de <code>if</code> y <code>foreach</code>	plantilla HTML
Escapado contra XSS	función <code>e()</code>

 **Amplía el proyecto:** añade ordenación (`?orden=precio_desc`), paginación con `array_slice` y un buscador por nombre con `str_contains`.

9. Errores típicos y cómo depurarlos

Síntoma	Causa probable	Solución
Pantalla en blanco	Error fatal con <code>display_errors</code> desactivado	Revisa el log de Apache/PHP
<code>Parse error: syntax error, unexpected ...</code>	Falta un <code>;</code> , <code>}</code> o <code>)</code>	Mira también la línea anterior
<code>Undefined variable / Undefined array key</code>	Variable o clave inexistente	<code>isset()</code> o <code>??</code>
<code>headers already sent</code>	Salida (incluso un espacio) antes de <code>header()</code>	Elimina HTML/espacios previos o usa <code>ob_start()</code>
El campo del formulario no llega	Falta el atributo <code>name</code>	Añade <code>name</code>
<code>if (\$a = 1)</code> siempre es verdadero	Asignación en lugar de comparación	Usa <code>===</code>
Bucle infinito	La variable de control no cambia	Revisa el incremento
<code>switch</code> ejecuta varios casos	Falta <code>break</code>	Añádalo o usa <code>match</code>
Caracteres raros (Ã±)	Codificación	UTF-8 en el fichero y en <code><meta charset></code>
<code>Call to undefined function</code>	Falta el <code>include/require</code> o error de nombre	Revisa la ruta y el nombre
El checkbox desmarcado «no existe»	Los campos sin marcar no se envían	Comprueba con <code>isset()</code> o <code>??</code>

Activar los errores en desarrollo (nunca en producción)

```
<?php
ini_set('display_errors', '1');
error_reporting(E_ALL);
```

Técnicas de depuración

```
<?php
echo '<pre>';
var_dump($variable);           // tipo y valor
print_r($array);               // estructura de un array
echo '</pre>';

error_log("Valor de x: " . $x); // escribe en el log del servidor
```

💡 Método práctico: **reduce el problema**. Comenta partes del código, imprime variables intermedias y comprueba en cada paso que el valor es el esperado.

10. Actividades y autoevaluación

Actividades prácticas (de menor a mayor dificultad)

1. **Par o impar:** formulario con un número; indica si es par o impar y positivo o negativo.
2. **Tabla de multiplicar** configurable por GET (`?n=7`), validando que `n` sea un entero entre 1 y 20.
3. **Notas:** array asociativo de alumnos y notas; muestra una tabla con la media, la nota máxima y los aprobados.
4. **Librería:** crea `funciones.php` con `esPrimo()`, `factorial()` y `fizzBuzz()` y úsalas desde otra página.
5. **Calculadora** por POST (+, -, ×, ÷) con `match`, controlando la división por cero.
6. **Registro con validación:** nombre, email, ciclo (lista blanca) y aficiones (array de checkbox), recordando lo escrito si hay errores.
7. **Reto:** *Adivina el número* usando un campo oculto para mantener el estado (sin sesiones).

Autoevaluación

1. ¿Qué diferencia hay entre `==` y `===`? Pon un ejemplo en el que den resultados distintos.
2. ¿Qué ocurre en un `switch` si olvidas un `break`?
3. ¿Cuándo usarías `do ... while` en lugar de `while`?
4. ¿Cómo recibe PHP varios checkbox con el mismo `name`?
5. ¿Por qué no debe usarse GET para un formulario de login?
6. ¿Qué diferencia hay entre `include` y `require_once`?
7. ¿Qué hace `htmlspecialchars()` y por qué ayuda a evitar XSS?
8. ¿Por qué no basta con validar en JavaScript?
9. ¿Qué imprime este código?

```
$a = [3, 1, 2];
sort($a);
foreach ($a as $k => $v) { echo "$k=$v "; }
```

10. ¿Qué devuelve `match (5) { "5" => 'a', default => 'b' }`?

► Soluciones

Resumen del tema

Concepto	Idea clave
Operadores	Comparación (<code>==</code> , <code>===</code> , <code><...</code>) y lógicos (<code>&&</code> , <code> </code> , <code>!</code>). Preferir <code>===</code> .
<code>if</code>	Dos flujos de ejecución; puede anidarse con <code>elseif</code> ; tiene forma abreviada (<code>? :</code>) y alternativa (<code>:</code> y <code>endif</code>).
<code>switch</code>	Selección múltiple según un valor; necesita <code>break</code> ; <code>default</code> opcional; <code>match</code> en PHP 8.
Bucles	<code>while</code> (indeterminado), <code>do...while</code> (al menos una vez), <code>for</code> (contador), <code>foreach</code> (arrays).
Arrays	Escalares (índice), asociativos (clave) y multidimensionales; más de 70 funciones.
Funciones	Devuelven valor (<code>return</code>); procedimientos no. Argumentos por valor, por referencia o con valor predeterminado.
Librerías	Funciones reutilizables en ficheros; <code>include/require</code> y variantes <code>_once</code> .
Funciones predefinidas	Cadenas, arrays, fecha y hora, matemáticas. Cuidado con las obsoletas.
GET	Recupera información; datos en la URL; visibles y limitados; <code>\$_GET</code> .
POST	Envía información; datos en el cuerpo; sin límite práctico; <code>\$_POST</code> .
Seguridad	Validar y escapar siempre los datos del cliente; HTTPS; respetar el RGPD.